Command-Line MiniScript

Command-line MiniScript is a stand-alone version of MiniScript 2.0 for use in a terminal. You can use it interactively, to try out ideas and do quick calculations, or to run MiniScript scripts much as you would Python or Perl scripts. On top of the standard MiniScript language, it adds intrinsics for working with files, running shell commands, reading the keyboard, and importing library modules. Standard library modules include advanced terminal support, enabling you to create sophisticated text-based UIs.

This document covers how to install and run command-line MiniScript, and the extra features it provides. For the language itself, see the MiniScript Manual; if you are coming from MiniScript 1.x, see also *What's New in MiniScript 2.0*. Both are included in the download.

CONTENTS

Installation 2
Running MiniScript 3
The interactive prompt (REPL) 3
Running a script file 4
Running code given on the command line 4
Running a script, then the REPL 4
Command-line options 5
The REPL 5
Input and output history 6
Line editing and metacommands 6
Interrupting 6
Command-line intrinsics 7
exec 7
file 8
RawData 9
key 10
Importing modules 11
The search path 11
Bundled modules 11
Environment variables 12
Changes from MiniScript 1.x 13

Installation

Download the archive for your platform from the releases page, and unpack it:

Platform	Archive	Executable
macOS (Intel and Apple Silicon)	`miniscript-mac.zip`	`miniscript`
Linux (x86-64)	`miniscript-linux.tar.gz`	`miniscript`
Windows (x86-64)	`miniscript-win.zip`	`miniscript.exe`
Other (requires .NET 6 or later)	`miniscript-csharp.zip`	`miniscript-csharp.dll`

Each archive unpacks to a single folder containing the executable, a `lib` folder of library modules, and the documentation. Place this folder wherever you like.

You can run MiniScript right from that folder. To run it from anywhere, either add the folder to your `PATH`, or (on macOS and Linux) put a symbolic link to the executable somewhere already on your `PATH`, such as `/usr/local/bin`:

```
ln -s /path/to/miniscript-mac/miniscript /usr/local/bin/miniscript
```

> **IMPORTANT**
>
> Keep the `lib` folder next to the executable. MiniScript looks there for library modules when you `import` them (see Importing modules). A symbolic link to the executable is fine; MiniScript follows it to find the real library folder.

The .NET build is a fallback for platforms not covered by the native builds. It is slower, and needs a .NET runtime installed; run it as `dotnet miniscript-csharp.dll`, with the same options described below.

Running MiniScript

There are three ways to use command-line MiniScript.

The interactive prompt (REPL)

Run `miniscript` with no arguments to start the REPL (read-eval-print loop):

```
$ miniscript
MiniScript 2.0
Build: C++ (goto) version, built Oct 12 2026 09:22:51
Enter !help for REPL help.

 _in[0]: x = 6 * 7

 _in[1]: x
_out[1]: 42
```

Type any MiniScript statement and press Return to run it. An expression on its own is evaluated and its value displayed. When you begin a block (`if`, `for`, `while`, `function`, etc.), the prompt changes to `...:` until the block is complete.

To quit, use `exit`, or press Control-D (on Windows, Control-Z then Return). See The REPL for more of what the REPL can do.

Running a script file

To run a script, give its path:

```
miniscript myscript.ms
```

The script runs, and MiniScript exits when it finishes. Anything after the script path is passed along to the script as arguments, available through `shellArgs`:

```
miniscript myscript.ms one two three
```

Here `shellArgs` would be `["myscript.ms", "one", "two", "three"]`: the script path comes first, followed by its arguments.

The standard extension for MiniScript source files is `.ms`.

On macOS and Linux, you can make a script directly executable by giving it a "shebang" line as its first line, and marking it executable with `chmod +x myscript.ms`:

```
#!/usr/bin/env miniscript
print "Hello from a script!"
```

MiniScript ignores a first line that begins with `#!`, so the script still works when run as `miniscript myscript.ms`.

Running code given on the command line

The `-c` option runs a bit of code given directly on the command line, then exits:

```
$ miniscript -c 'print 6 * 7'
42
```

Separate multiple statements with semicolons. Anything after the code is passed to it as arguments, as with a script; in place of the script path, `shellArgs[0]` is `"-c"`.

Running a script, then the REPL

Add `-i` to drop into the REPL after a script (or `-c` code) finishes, with all its globals still in place. This is handy for inspecting a script's results, or for loading up some functions to play with:

```
miniscript -i myfunctions.ms
```

In this mode, `exit` in the script ends just the script, not the REPL.

Command-line options

```
miniscript [options] [script.ms [args...]]
```

The following options are of use to MiniScript users:

Option	Description
`-c CODE`	Run `CODE` directly instead of a script file. Everything after `CODE` is passed to it as arguments.
`-i`, `--interactive`	Enter the REPL after running the script or `-c` code.
`-q`, `--quiet`	Suppress the startup banner in the REPL.
`-h`, `-?`, `--help`	Show a summary of options, and exit.
`-v`, `--version`	Show version information, and exit.
`--`	End of options. Use this before a script whose name begins with a dash.

...while these ones are of use mainly to the developers of MiniScript itself:

Option	Description
`-d`, `--debug`	Print diagnostic detail (compiled code, register contents, etc.) while compiling and running.
`--vis`	Run with a full-screen visualization of the virtual machine.
`--test`	Run the built-in unit and integration test suites.

Options must come before the script path (or the `-c` code). Everything after that is passed to the script, even if it looks like an option. Single-letter options may be combined; for example, `-qi` is the same as `-q -i`.

With no script and no `-c`, MiniScript starts the REPL.

The REPL

Each input in the REPL is numbered, and the prompt shows the number: `_in[3]:`. When an input is an expression, its value is shown with the matching number: `_out[3]: 42`.

Input and output history

The REPL keeps every input and output, available as two global lists:

- **`_in`** — the text of each input, in order. `_in[3]` is the input numbered 3.
- **`_out`** — the value of each input that was a bare expression. Inputs that produced no output have `null` here.

So you can reuse an earlier result in a new calculation:

```
 _in[4]: _out[1] / 2
_out[4]: 21
```

`reset` clears all global variables, and both history lists, giving you a fresh start without leaving the REPL.

Line editing and metacommands

The REPL supports line editing: use the left and right arrow keys to move within a line, and the up and down arrow keys to step through previous inputs.

A line beginning with `!` is a *metacommand*, handled by the REPL itself rather than run as MiniScript:

Metacommand	Effect
`!help`	Show the list of metacommands.
`!?`	Show recent input history.
`!? N`	Show the last `N` history entries.
`!? word`	Show history entries containing `word`.
`!? N word`	Show the last `N` history entries containing `word`.
`!N`	Run input number `N` again.
`!-N`	Run the `N`th most recent input again.

Interrupting

Press Control-C to stop a running program and return to the prompt. At the prompt, Control-C throws away any partly entered (multi-line) input.

Command-line intrinsics

Command-line MiniScript adds the following intrinsics to the standard set documented in the MiniScript Manual.

Intrinsic	Description
`shellArgs`	A list of the command-line arguments, starting with the script path (or `"-c"`, for code run with `-c`), followed by the arguments given after it. The list is frozen (read-only).
`env`	A map of environment variables. Changes you make to it are passed on to commands run with `exec`.
`exec(cmd, timeout=30)`	Run a shell command; see `exec`.
`exit(resultCode=0)`	Stop the program, returning `resultCode` as the process exit status.
`import libname`	Load a library module; see Importing modules.
`file`	A module of file and directory functions; see `file`.
`FileHandle`	The class of open files returned by `file.open`.
`RawData`	A class representing a buffer of binary data; see `RawData`.
`key`	Single-keystroke keyboard input; see `key`.
`gc`	Access to the garbage collector: `gc.collect` runs a full collection, and `gc.stats` returns a map of statistics.
`_in`, `_out`, `reset`	REPL history and reset; see The REPL.

`exec`

`exec(cmd, timeout=30)` runs `cmd` with the system shell, waits for it to finish, and returns a map with three entries:

- `output` — what the command wrote to standard output
- `errors` — what the command wrote to standard error
- `status` — the command's exit status (0 usually means success)

```
r = exec("ls -l")
if r.status == 0 then print r.output else print "Error: " + r.errors
```

If the command runs longer than `timeout` seconds, it is stopped; the result then has status 124, and `errors` ends with "Timed out".

`file`

The `file` module works with files and directories. Paths may be absolute, or relative to the current working directory.

Function	Description
`file.curdir`	Return the current working directory.
`file.setdir(path)`	Change the current working directory. You can also write `file.curdir = path`.
`file.makedir(path)`	Create a new directory.
`file.children(path)`	Return a list of the names of the files and directories within the directory `path`.
`file.name(path)`	Return just the file name from `path` (the part after the last slash).
`file.parent(path)`	Return the directory part of `path`.
`file.child(parentPath, childName)`	Join a directory path and a name.
`file.exists(path)`	Return 1 if a file or directory exists at `path`, else 0.
`file.info(path)`	Return a map with `path` (absolute), `isDirectory`, `size` (in bytes) and `date` (modification time), or `null` if there is no such file.
`file.copy(oldPath, newPath)`	Copy a file.
`file.move(oldPath, newPath)`	Move or rename a file.
`file.delete(path)`	Delete a file, or an empty directory.
`file.readLines(path)`	Return the contents of a text file as a list of strings, one per line.
`file.writeLines(path, lines)`	Write a list of strings to a text file, one per line, replacing any previous contents.
`file.loadRaw(path)`	Return the contents of a file as a `RawData` object.
`file.saveRaw(path, data)`	Write a `RawData` object to a file.
`file.open(path, mode="r+")`	Open a file, returning a `FileHandle`.

Functions that fail return an error value (see *What's New in MiniScript 2.0*), so a failure on a line of its own stops the program with a message, while code like `lines = file.readLines(path)` can check for the error itself.

The `mode` for `file.open` is one of:

Mode	Meaning
`"r"`	Read only. The file must already exist.
`"w"`	Write only. Creates the file, or empties it if it exists.
`"a"`	Append: write only, adding to the end of the file.
`"r+"`	Read and write, creating the file if needed (the default).

A `FileHandle` has these methods:

Method	Description
`write(data)`	Write a string to the file.
`writeLine(data)`	Write a string, followed by a line break.
`read(byteCount=-1)`	Read up to `byteCount` bytes as a string; with no argument, read to the end of the file.
`readLine`	Read the next line, without its line break.
`atEnd`	Return 1 if the read position is at the end of the file.
`position`	Return the current read/write position, in bytes from the start. You can also assign to it: `f.position = 0`.
`seek(pos)`	Move to the given position.
`isOpen`	Return 1 if the file is still open.
`close`	Close the file.

```
f = file.open("notes.txt", "w")
f.writeLine "Remember the milk."
f.close
print file.readLines("notes.txt")
```

`RawData`

A `RawData` object is a resizable buffer of bytes, for reading and writing binary files. Create one with `new RawData` and set its size with `resize(bytes)`, or get one from `file.loadRaw`.

Member	Description
`len`	The size of the buffer, in bytes (assignable).
`resize(bytes=32)`	Change the size of the buffer.
`littleEndian`	Byte order for multi-byte values: 1 (the default) for little-endian, 0 for big-endian.
`byte(offset)`, `setByte(offset, value)`	Unsigned 8-bit integer
`sbyte(offset)`, `setSbyte(offset, value)`	Signed 8-bit integer
`ushort(offset)`, `setUshort(offset, value)`	Unsigned 16-bit integer
`short(offset)`, `setShort(offset, value)`	Signed 16-bit integer
`uint(offset)`, `setUint(offset, value)`	Unsigned 32-bit integer
`int(offset)`, `setInt(offset, value)`	Signed 32-bit integer
`float(offset)`, `setFloat(offset, value)`	32-bit floating-point number
`double(offset)`, `setDouble(offset, value)`	64-bit floating-point number
`utf8(offset, bytes=-1)`	Decode UTF-8 text starting at `offset`; with no `bytes`, read to the end.
`setUtf8(offset, value)`	Store a string as UTF-8, returning the number of bytes written.

`key`

The `key` module reads the keyboard one keystroke at a time, without waiting for Return, as needed for games and other interactive console programs.

Member	Description
`key.available`	Return 1 if a keystroke is waiting to be read, else 0. Does not wait.
`key.get`	Wait for the next keystroke and return it as a one-character string.
`key.raw`	A flag, normally 0. When 0, arrow keys and other special keys are translated into single characters with the same codes on every platform. Set it to 1 to get the raw bytes the terminal sends instead.

Keys read this way are not echoed to the screen; print them yourself if you want them shown. The terminal is switched into single-keystroke mode when you first use `key`, and restored automatically when the program ends or uses `input`. The bundled `vt` module, which provides codes for colors and cursor control, pairs well with `key`.

Importing modules

`import` loads a MiniScript source file and makes its contents available as a map:

```
import "listUtil"
print [3, 1, 4, 1, 5].max
```

`import "foo"` looks for a file named `foo.ms`, runs it, and stores the variables it defined in a global variable named `foo`. (You can also write `foo = import("foo")`, to choose a different variable name or check for an error.) Many modules, like `listUtil`, also add methods to the built-in types, so you can use them without going through the module variable at all.

The search path

`import` by default looks for modules in these places, in order:

1. the folder containing the script being run (or the current directory, for the REPL and `-c`)
2. a `lib` folder inside that folder
3. the `lib` folder next to the MiniScript executable

To change this, set the `MS_IMPORT_PATH` environment variable to a list of folders separated by `:` or `;`. Entries may refer to other environment variables as `$NAME` or `${NAME}`; the default value is:

```
$MS_SCRIPT_DIR:$MS_SCRIPT_DIR/lib:$MS_EXE_DIR/lib
```

You can set `MS_IMPORT_PATH` in your shell before running MiniScript, or change it from within a script, through `env`:

```
env.MS_IMPORT_PATH = env.MS_IMPORT_PATH + ":/my/modules"
```

Bundled modules

The `lib` folder contains these modules:

Module	Description
`dateTime`	Get the current date and time, and convert dates between strings and numbers.
`grfon`	Read and write GRFON, a simple, human-friendly data file format.
`importUtil`	`ensureImport`, which imports a module only if it has not been imported already.
`json`	Read and write JSON data.
`listUtil`	Additional methods on lists (searching, sorting, statistics, and more).
`mapUtil`	Additional methods on maps.
`mathUtil`	Additional math functions and constants.
`matrixUtil`	Functions for working with 2D arrays of numbers.
`qa`	Assertions and other quality-assurance helpers.
`stringUtil`	Additional methods on strings.
`tsv`	Read tab-separated value (TSV) files, such as spreadsheet exports.
`vt`	Codes for controlling a VT100-compatible terminal: colors, cursor movement, clearing the screen, and more.

Each module begins with comments describing what it provides, so the best reference for a module is its own source file.

Environment variables

These environment variables affect command-line MiniScript, or are set by it for scripts to use.

Variable	Description
`MS_IMPORT_PATH`	The folders `import` searches for modules. See The search path.
`MS_SCRIPT_DIR`	Set by MiniScript to the folder containing the script being run (or the current directory, when there is no script).
`MS_EXE_DIR`	Set by MiniScript to the folder containing the MiniScript executable.

All environment variables are available to scripts through the `env` map.

Changes from MiniScript 1.x

Command-line MiniScript 2.0 works much like 1.x, with a few differences:

- **Code run with `-c` can take arguments.** Anything after the code is available in `shellArgs`, after a first element of `"-c"`.
- **New intrinsics:** `key`, `gc`, `RawData`, `file.loadRaw` and `file.saveRaw`, and the REPL's `_in`, `_out` and `reset`.
- **Errors are values.** Functions such as `file.readLines` and `file.open` return an error value on failure, rather than `null`.
- **The REPL** numbers its inputs and outputs, keeps them in `_in` and `_out`, and adds history metacommands such as `!?` and `!N`.